

COMP167 - Major Programming Assignment 2 – Fall 2025

1 Introduction

This assignment requires you to create a Java application that will be used to inventory mowers in a hardware store. In addition to the classes that store the mower data, you will design a GUI to manage the inventory.

2 Classes

You are required to implement the following classes at a minimum. You may add other classes (and methods) if you need them.

2.1 Engine Class

Engine	
-manufacturer : String -horsePower : double -cylinders : integer	Engine manufacturer Horse power of engine Number of cylinders
+Engine() +Engine(//all 3 Engine properties) +//Accessor and Mutators +toString():String	Put each property on a separate line.

2.2 Mower Class (Abstract)

Mower	
-manufacturer : String -year : integer -serialNumber : String	Mower manufacturer Year of manufacture Serial number of mower
+Mower() +Mower(//All properties) +cut() : void +//Getters and setters +toString()	(abstract) The procedure used to cut grass/vegetation. Put each property on a separate line.

2.3 LawnTractor Class

This class will define rideable lawn mower.

LawnTractor	
-engine : Engine -model : String -deckWidth : double	Mower engine Model of Lawn Tractor Width of mower deck
+LawnTractor() +//Accessor and Mutators +toString():String	Put each property on a separate line.

2.4 CommercialMower Class

This class will define larger and more durable lawn tractors.

CommercialMower	
-operatingHours : double -zeroTurnRadius : boolean	Total operating hours of the mower Is this a zero-turn radius mower?
+CommercialMower() +CommercialMower(opHrs: double, zeroTurnRad : boolean) +//Accessor and Mutators +toString():String	Put each property on a separate line.

2.5 WalkBehindMower Class (Abstract)

Mowers where the operator guides the mower while walking behind the mower.

WalkBehindMower	
-cutWidth : double -wheelDiameter : double	Blade width of mower Diameter of the mower wheels
+WalkBehind() +//Accessor and Mutators +toString():String	Put each property on a separate line.

2.6 RotaryMower

RotaryMower	
-engine : Engine -selfPropelled : boolean	Mower engine Is the mower self-propelled
GasPowered() +//Accessor and Mutators +toString():String	Put each property on a separate line.

2.7 PushReelMower

PushReelMower	
-numBlades : integer	Number of cutting blades on the reel.
+PushReelMower() +//Accessor and Mutators +toString():String	Put each property on a separate line.

2.8 MowerWareHouse

MowerWareHouse	
-storeName : String -mowers : ArrayList<Mower>	Name of the mower store
+MowerWareHouse() +//Accessor and Mutators +readMowerData(inputFileName : String) : void +saveMowerData(outputFileName :String):void +toString():String	Put each property on a separate line.

Handling ArrayLists

Each ArrayList should have five associated methods to perform: getNum, add, remove, get and set. So if you have an ArrayList named *widgets* that stored items of type *Widget*, then the associated UML behaviors would be:

```
+getWidgetSize() : int //Return the number of items in the ArrayList widgets.  
+getWidget(index:int) : Widget //get the Widget at location index in ArrayList widgets  
+setWidget(index:int, item:Widget):void //store item at location index in the ArrayList  
widgets.  
+addWidget(item:Widget):void //Append the Widget to the  
ArrayList.  
+removeWidget( index:int ) : Widget //remove and return the  
Widget at location index
```

3 Input File

The name of the input file will be supplied using command-line arguments. If no command-line argument is supplied, then your program should prompt the user for the input file using the *JFileChooser* class. Figure 1 has the format of the input file:

Store name
Mower Class Properties
Mower Subclass Type (L, C, G, P)
Subclass Properties

Note: All properties will be on a separate line in the same order listed in the UML diagrams.

Figure 1: Input File Format

4 Output File

The format for the output file should be identical to that of the input file. In other words, after writing your output file, you should be able to read it back in as an input file. The `toString()` methods of your classes are designed to make file output simple.

5 Graphical User Interface

Use the IntelliJ SceneBuilder application to design a GUI for your application. The GUI will not be functional – This is just a design exercise. Include a description detailing how your GUI would function if the buttons and other controls worked. Your description should be detailed enough so that another Java programmer would have enough information to implement all the functionality you have designed for your GUI. Create a GUI folder in your project and include all .fxml files and your GUI description.

6 Grading

If your project does not compile, it receives a grade of zero. If you do not document your program according to the documentation guidelines, the graders have been instructed to deduct **up to 25%**.

Level 1 (10%): Create UML domain diagram that shows how all the classes are related. There will be a submission link on BlackBoard for the .pdf file containing your UML diagram (**Due: 10/14/2025**)

Level 2 (45%): Implement all the classes **except** the MowerWareHouse class.

Level 3 (70%): Implement the MowerWareHouse class except the `readMowerData()` and `saveMowerData()` methods. Use the main method to create one of each type of concrete class and add them to the ArrayList. Output the contents of the MowerWareHouse object to a JOptionPane using the `toString()` method of the MowerWareHouse class.

Level 3 (90%): Modify your main so that it uses command-line arguments to provide the inputfile name. Add the logic to obtain the file name from a JFileChooser if no command-line argument is provided. Implement the `readMowerData` and `saveMowerData` methods. To prove your code works, read the input file and add additional mowers by creating objects in the main method and adding them to the MowerWareHouse object. Save the updated MowerWareHouse object using the `saveMowerData()` method. When your testing/evaluation of your code is complete, you should have every type of mower in our input file to prove your classes are implemented correctly. You are encouraged to share your input files with classmates for additional testing/evaluation of your code. A percentage of the Level 3 grade will be based on the final input file that you submit with your code.

Level 4 (100%): Implement the GUI design.

7 Submission

Unless otherwise posted, send a push notification to your TA when you have completed your assignment.